

LG Soft India

Advanced Programming Test Cases & Edge-Case Analysis

8 Programming Problems • 120+ Difficult / Corner / Edge Test Cases

Purpose

This document expands the programming-question PDF into an interview-oriented test suite. Each problem contains the original question, the source sample, at least 15 deliberately difficult test cases, expected results or validation conditions, and the key implementation traps those cases are designed to expose.

Source: LG Soft India — All Programming Questions (uploaded PDF).

The source PDF states that there are 8 programming questions.

How to Use This Test Suite

- Do not use only the sample input/output to judge a solution. The difficult cases below are designed to expose hidden assumptions.
- For interview evaluation, first check functional correctness, then deliberately run the duplicate, tie, overflow, frequency, and boundary cases.
- For problems with multiple valid outputs, validate the required property rather than comparing the candidate's output to one exact string.
- The test suite emphasizes corner cases and adversarial inputs rather than ordinary happy-path examples.
- The source PDF explicitly requires $O(n)$ for Longest Consecutive Sequence; complexity should therefore be evaluated in addition to output correctness.

High-Risk Traps Across the 8 Problems

Problem	Primary Trap
Count and Say	Treating the sequence as numbers instead of strings; inefficient/fixed-size output handling.
Closest Pair Sum	Missing the larger-sum tie-breaker; integer overflow.
Isomorphic Strings	Checking only source→target and forgetting target uniqueness.
Missing Element	Duplicate values; confusing 'missing value' with 'value absent entirely'.
Longest Consecutive Sequence	Using sorting instead of the required $O(n)$ approach; duplicates and integer boundaries.
Backspace String Compare	Consecutive backspaces and backspaces when nothing exists to delete.
Reorganize String	Failure to detect impossible frequency distributions; incorrect greedy choice.
Task Scheduler	Incorrect idle-slot formula; repeated maximum-frequency tasks; zero cooldown.

1. Count and Say

Source question: Given a positive integer n , return the n th term of the count-and-say sequence. The sequence starts with "1"; each next term is formed by reading consecutive runs, counting each run, and then stating count followed by digit.

Source sample: $n = 4 \rightarrow "1211"$.

Source: [LeetCode #38](#) (link stated in the uploaded PDF).

Advanced Test Cases

#	Input n	Expected Output / Condition
1	1	1
2	2	11
3	3	21
4	4	1211
5	5	111221
6	6	312211
7	7	13112221
8	8	1113213211
9	9	31131211131221
10	10	13211311123113112211
11	11	11131221133112132113212221
12	12	3113112221232112111312211312113211
13	15	311311222113111231131112132112311321322112111312211312111322212311322113212221
14	20	Valid count-and-say term; expected length = 302
15	30	Valid count-and-say term; stress test for output growth, memory and time

Why These Cases Are Difficult

1. $n = 1$ checks correct initialization of the sequence.
2. $n = 2-4$ catches basic generation and run-counting errors.
3. $n = 10-15$ exposes incorrect grouping of consecutive runs and string-building mistakes.
4. $n = 20$ stresses output growth and fixed-size buffers.
5. $n = 30$ is a stress case for memory use and inefficient repeated string concatenation.
6. $n = 0$ is an invalid-input boundary because the source explicitly specifies a positive integer.

Killer Point

A solution that stores terms in an integer type is fundamentally unsafe. The sequence rapidly becomes a string that cannot be represented by ordinary integer types.

2. Absolute Pair Sum Closest to Zero

Source question: Given an array containing positive and negative integers, find the pair whose sum is closest to zero. If two pairs are equally close, prefer the pair with the larger sum.

Source sample: $[-8, 5, 2, -6] \rightarrow -1$, from pair $(5, -6)$.

Source: *GeeksforGeeks (as stated in the uploaded PDF).*

#	Input Array	Expected Result / Condition
1	$[-8, 5, 2, -6]$	$-1 \rightarrow (5, -6)$
2	$[-1, 1]$	0
3	$[0, 0, 5, -5]$	0
4	$[1, 2, 3, 4, 5]$	$3 \rightarrow (1, 2)$
5	$[-1, -2, -3, -4, -5]$	$-3 \rightarrow (-1, -2)$
6	$[-3, 1, 1]$	$+2 \rightarrow (1, 1)$, tie wins by larger sum
7	$[-10, 4, 6]$	$-4 \rightarrow (-10, 6)$
8	$[-100, -1, 2, 50]$	$1 \rightarrow (-1, 2)$
9	$[-5, -5, 10, 10]$	0
10	$[-100, 99, 1]$	$-1 \rightarrow (-100, 99)$
11	$[-2147483648, 2147483647, 1]$	$-1 \rightarrow (-2147483648, 2147483647)$
12	$[-2147483648, -1, 2147483647]$	-1
13	$[-7, 5, 2]$	$-2 \rightarrow (-7, 5)$
14	$[-10, -3, 3, 10]$	0
15	$[-1000000000, 999999999, 2, -1]$	$-1 \rightarrow (-1000000000, 999999999)$

Case-by-Case Traps

7. Case 3 checks multiple zero-producing pairs and duplicate zero values.
8. Case 6 is a direct tie-break test: $-3+1 = -2$ and $1+1 = +2$; both have absolute value 2, so +2 must win.
9. Cases 11 and 12 use 32-bit integer boundaries and can expose unsafe arithmetic.
10. Case 14 checks whether the implementation handles exact cancellation when values appear symmetrically.
11. Case 15 combines large magnitudes with a near-zero result.

Killer Point — Tie Breaking

Do not select the first pair with minimum absolute sum. When absolute sums are equal, the pair with the numerically larger sum must be selected.

Killer Point — Overflow

Use a sufficiently wide arithmetic type when adding extreme integer values. A careless 32-bit implementation can overflow while evaluating candidate sums.

3. Isomorphic Strings (Couple Strings)

Source question: Determine whether characters of the first string can be mapped one-to-one onto the second string, with a consistent mapping and no two source characters mapping to the same target character.

Source sample: "paper", "title" → YES.

Source: LeetCode #205 (as stated in the uploaded PDF).

#	s	t	Expected
1	paper	title	YES
2	egg	add	YES
3	foo	bar	NO
4	ab	aa	NO
5	aa	ab	NO
6	abc	def	YES
7	abc	dee	NO
8	aaaa	bbbb	YES
9	abab	cdcd	YES
10	abab	cddc	NO
11	abcabc	xyzxyz	YES
12	abcde	aabcd	NO
13	aab	xyy	YES
14	aab	xyy	NO
15	abcabcabc	xyzxyzxyz	YES

Case-by-Case Traps

- Case 4 catches two different source characters mapping to the same target character.
- Case 5 catches implementations that only check whether each source character maps consistently, but fail to enforce target uniqueness.
- Case 7 contains $b \rightarrow e$ and $c \rightarrow e$, which violates the one-to-one requirement.
- Case 10 preserves repeated structure in the source but changes the structural relationship in the target.
- Case 14 creates a conflict because the repeated source character would need two different target mappings.
- Different-length strings must be rejected. If empty strings are permitted by the implementation contract, two empty strings are normally equivalent.

Killer Point

A correct solution needs two-way uniqueness: source→target consistency AND target→source uniqueness. A single hash map is not enough unless the implementation separately guarantees the reverse constraint.

4. Missing Element (Array N and N-1)

Source Ambiguity to Clarify

The uploaded PDF says the second array contains N-1 elements formed by removing exactly one element, but its sample shows Array1=[2,1,2,1] and Array2=[1], which does not have N-1 elements. The test cases below follow the written N/N-1 definition.

Source question: Given two arrays, the second formed by removing exactly one element from the first, find the missing element.

Source sample exactly as printed: Array1=[2,1,2,1], Array2=[1] → 2.

Source: GeeksforGeeks (as stated in the uploaded PDF).

#	Array 1	Array 2	Missing
1	[1,2,3,4]	[1,2,4]	3
2	[1,2,3,4]	[2,3,4]	1

3	[1,2,3,4]	[1,2,3]	4
4	[2,2,3,4]	[2,3,4]	2
5	[1,1,1,2]	[1,1,2]	1
6	[-5,-4,-3,-2]	[-5,-3,-2]	-4
7	[-10,0,10,20]	[-10,10,20]	0
8	[100,200,300,400]	[100,300,400]	200
9	[5,5,5,5,5]	[5,5,5,5]	5
10	[1,2,2,3,3,3]	[1,2,2,3,3]	3
11	[-1,-1,0,0,1,1]	[-1,0,0,1,1]	-1
12	[2147483647,0,-2147483648]	[0,-2147483648]	2147483647
13	[1000000000,-1000000000,5]	[-1000000000,5]	1000000000
14	[7,8,7,9,8,9]	[7,8,9,8,9]	7
15	[0,0,0,0,1,1,1]	[0,0,0,1,1,1]	0

Case-by-Case Traps

18. Cases 4, 5, 9, 10, 11, 14 and 15 are duplicate-heavy and expose solutions that only search for a value absent from Array 2.
19. Case 6 checks negative values.
20. Case 7 checks zero.
21. Cases 12 and 13 check large positive/negative values.
22. Case 14 is especially useful because every distinct value remains in Array 2, but one occurrence is missing.
23. The correct reasoning is occurrence-based: Array 2 contains every occurrence except exactly one occurrence from Array 1.

5. Longest Consecutive Sequence

Source question: Given an unsorted array, find the length of the longest run of consecutive integers. The source explicitly requires an O(n) solution.

Source sample: [100,4,200,1,3,2] → 4.

Source: LeetCode #128 (as stated in the uploaded PDF).

#	Input	Expected Length
1	[100,4,200,1,3,2]	4
2	[1]	1
3	[]	0
4	[1,2,3,4,5,6,7,8,9,10]	10
5	[10,9,8,7,6,5,4,3,2,1]	10
6	[1,1,1,2,2,3,3]	3
7	[0,-1,-2,-3,-4,10]	5
8	[-5,-4,-3,-2,-1,0,1]	7
9	[100,4,200,1,3,2,2,3,4]	4
10	[1,3,5,7,9]	1
11	[10,11,12,14,15,16,17,100]	4
12	[-2147483648,-2147483647,-2147483646]	3
13	[2147483645,2147483646,2147483647]	3
14	[-2,-1,0,1,2,3,100,101]	6
15	[1000000000,999999999,1000000001,999999998,5]	4

Case-by-Case Traps

24. Case 3 checks empty-input handling.
25. Cases 4 and 5 check long runs in sorted and reverse-sorted order.
26. Cases 6 and 9 verify that duplicates do not artificially increase or break the sequence.
27. Cases 7, 8 and 14 check negative-to-positive consecutive ranges.

28. Cases 12 and 13 sit at 32-bit integer boundaries.
29. Case 15 is deliberately unordered and contains a consecutive run near one billion.
30. The source requires $O(n)$; a sort-based $O(n \log n)$ solution may produce correct outputs but does not satisfy the stated complexity requirement.

Killer Point — Complexity

For a candidate claiming $O(n)$, ask how each number is prevented from causing repeated full scans. Hash-set based boundary detection is a common route; sorting is not $O(n)$.

6. Backspace String Compare

Source question: Strings may contain '#', where '#' represents a backspace deleting the previous character. Determine whether the two strings become equal after applying all backspaces.

Source sample: `s="ab#c", t="ad#c" → TRUE`.

Source: LeetCode #844 (as stated in the uploaded PDF).

#	s	t	Expected
1	ab#c	ad#c	TRUE
2	ab##	c#d#	TRUE
3	a#c	c	TRUE
4	a##c	c	TRUE
5	##		TRUE
6	###		TRUE
7	abc###		TRUE
8	abc###	#	TRUE
9	abc#d	abd	TRUE
10	abc##d	ad	TRUE
11	a##b	b	TRUE
12	a#b#c#		TRUE
13	ab#	a	TRUE
14	ab##c	c	TRUE
15	a#b	b	TRUE

Case-by-Case Traps

31. Cases 5 and 6 contain only backspaces and empty results.
32. Case 8 compares a completely erased string against a single '#', which is also completely erased.
33. Case 10 contains multiple consecutive backspaces in the middle of the string.
34. Case 11 erases both characters before producing b.
35. Case 12 repeatedly creates and erases characters and finishes empty.
36. Case 15 checks that a backspace can remove a character and then a new character can be appended.

Killer Point

A backspace at the beginning of a string has no effect. Any implementation that blindly decrements an index without guarding the boundary can fail on '###', '#a', or similar inputs.

7. Card Shuffle — Reorganize String

Source question: Rearrange a string so that no two adjacent characters are the same. Return any valid rearrangement, or an empty string if no valid rearrangement is possible.

Source sample: `"aab" → "aba"`.

Source: LeetCode #767 (as stated in the uploaded PDF).

#	Input	Expected Output / Condition
1	aab	Valid: e.g. aba
2	aa	""
3	aaa	""
4	a	a
5	ab	ab or ba
6	abc	Any permutation with no adjacent duplicate
7	aabb	Valid: abab or baba
8	aaab	""
9	aaabb	Valid: ababa
10	aaabc	Valid arrangement exists
11	aaaabc	""
12	aabbcc	Valid arrangement exists
13	aaaaabbbbcc	""
14	aaaaabbbbb	Valid arrangement exists
15	aaaaabbbbcb	Valid arrangement exists

Case-by-Case Traps

37. Cases 2 and 3 are impossible because one character dominates the entire string.
38. Case 8 has length 4 and max frequency 3; because $3 > \text{ceil}(4/2)=2$, no solution exists.
39. Case 9 is exactly balanced enough to alternate the dominant character: aaabb → ababa.
40. Case 13 has max frequency 5 in a length-11 string, so $5 \leq \text{ceil}(11/2)=6$; therefore this case is NOT automatically impossible. The actual counts must be evaluated.
41. Case 14 is perfectly balanced: five a's and five b's, so an alternating arrangement exists.
42. Case 15 has five a's, five b's and one c. A valid arrangement exists; this is a useful test against overly conservative impossibility checks.

Killer Point — Feasibility Rule

A necessary feasibility check is $\text{max_frequency} \leq \text{ceil}(N/2)$. If the maximum character count exceeds $\text{ceil}(N/2)$, no valid arrangement can exist. When it does not exceed the bound, the implementation still has to construct a valid arrangement.

8. Card Shuffle — Task Scheduler

Source question: Given tasks and a cooldown n, find the minimum number of time units needed to complete all tasks. The same task must be separated by at least n intervals; idle slots are allowed.

Source sample: [A,A,A,B,B,B], n=2 → 8.

Source: LeetCode #621 (as stated in the uploaded PDF).

#	Tasks	Cooldown n	Expected Minimum Time
1	[A,A,A,B,B,B]	2	8
2	[A,A,A]	2	7
3	[A,A,A]	0	3
4	[A,B,C]	2	3
5	[A,A,B]	2	4
6	[A,A,B,B]	2	5
7	[A,A,B,B,C]	2	5
8	[A,A,A,B,C]	2	7
9	[A,A,A,A,B,C]	2	10
10	[A,A,A,A,B,B,C,C]	2	10
11	[A,A,A,A,B,C,D]	3	13
12	[A,B,C,D,E]	10	5
13	[A,A,B,B,C,C]	2	6
14	[A,A,A,B,B,B,C,C]	2	8
15	[A,A,A,A,B,B,C,C,D]	2	13

Case-by-Case Traps

- 43. Case 1 is the canonical cooldown example: A B idle A B idle A B = 8.
- 44. Case 3 checks $n=0$; there should be no idle time.
- 45. Case 4 checks that cooldown is irrelevant when every task is unique.
- 46. Case 7 has three task types and can be scheduled without unnecessary idle slots.
- 47. Case 9 has a dominant task and exposes under-counted idle time.
- 48. Case 10 contains multiple high-frequency task types and tests whether the formula accounts for the final frame correctly.
- 49. Case 12 has a very large cooldown but no repeated task; the answer remains the number of tasks.
- 50. Case 13 is a useful tie-frequency case: A,B,C each occur twice and can be arranged as ABCABC, giving 6.
- 51. Case 15 combines a dominant task with several supporting tasks and a non-trivial cooldown.

Killer Point — Idle Slots

The common mistake is to insert cooldown after the final occurrence of the dominant task. The final task occurrence does not require trailing idle time. When multiple tasks share the maximum frequency, they can fill the final positions of the schedule frame.

Final Interview Evaluation Checklist

Area	What to Check	Failure Signal
Correctness	Passes ordinary + adversarial tests	Only sample cases pass
Boundary Handling	Empty, singleton, extremes, beginning/end conditions	Crashes or undefined behavior
Duplicates	Frequency/occurrence logic is correct	Uses set-only reasoning where counts matter
Tie Handling	Explicit tie-breaking rules respected	Returns first minimum or arbitrary wrong tie
Complexity	Meets stated $O(n)$ where required	Uses unnecessary $O(n \log n)$ / $O(n^2)$
Overflow	Uses safe arithmetic for extreme integers	32-bit overflow
Multiple Outputs	Checks validity constraints, not one exact arrangement	Rejects valid alternate output
Impossible Cases	Correctly detects when no arrangement exists	Loops forever / returns invalid string

Recommended Difficulty Labels

Difficulty	Meaning
Basic	Validates the core algorithm on small or direct cases.
Hard	Introduces duplicates, repeated structure, ties, or non-trivial boundaries.
Killer / Edge	Designed to expose incorrect assumptions, overflow, impossible configurations, or complexity violations.

Important Source Note

This document is based on the uploaded LG Soft India programming-question PDF. Where the PDF's Question 4 sample conflicts with its own $N/N-1$ definition, that inconsistency has been explicitly preserved and flagged rather than silently rewritten. The Task Scheduler Case 13 in this document has been checked against the stated scheduling objective and corrected to 6.